

minicache.

Um page cache multithread em C11, indexado por uma árvore splay auto-ajustável.

O caminho da apresentação

01

Introdução

o problema, a hipótese e o que o projeto entrega

02

Estruturas de dados

a splay em código: rotações, lookup que mede, despejo por profundidade

03

Sistemas operacionais

paginação, page fault, dirty bit, swap, threads e mutex — o curso no código

04

Arquitetura

os cinco módulos e a disciplina de engenharia

05

Resultados

hit ratio, profundidade de acesso e vazão — medidos

Disco é lento. O cache decide o que importa.

Capacidade finita

O cache guarda só uma fração das páginas do arquivo. Quando enche, alguém precisa sair antes da próxima entrada.

Duas políticas

O LRU clássico despeja o menos usado recentemente. A splay despeja a folha mais profunda — fria por construção.

A hipótese

O auto-ajuste da splay captura localidade temporal melhor que o LRU em cargas com acesso concentrado.

O minicache responde com medição real: **hit ratio, profundidade de acesso e vazão.**

Objetivos e entregas

O que o projeto entrega

- Page cache em C11 com índice splay e baseline LRU
- Comparativo experimental: carga uniforme × zipfiana
- Multithread com thread-safety comprovada
- Zero warnings, zero leaks, zero data races
- Benchmark e gráficos gerados por make bench / plots

Árvore splay

Um índice pageno → frame que se reorganiza a cada acesso.

Splay no acesso — todo lookup rotaciona o nó acessado até a raiz.

Localidade temporal — páginas quentes ficam perto do topo.

$O(\log n)$ amortizado — teorema de acesso de Sleator & Tarjan.

o nó — src/splay.c

```
typedef struct snode {
    uint64_t pageno;    int frame_id;
    struct snode *left, *right, *parent;
} snode;
```

splay_node — src/splay.c

```
static void splay_node(splay_t *t,
                      snode *x) {
    while (x->parent) {
        snode *p = x->parent;
        snode *g = p->parent;
        if (!g) {
            rotate(t, x);          /* zig */
        } else if ((g->left == p)
                   == (p->left == x)) {
            rotate(t, p);          /* zig-zig */
            rotate(t, x);
        } else {
            rotate(t, x);          /* zig-zag */
            rotate(t, x);
        }
    }
}
```


As duas particularidades, em código

01 A leitura que mede — e muda

```
splay_lookup — src/splay.c

while (cur) {
    if (pageno == cur->pageno) {
        *out_depth = depth; /* ANTES */
        int fid = cur->frame_id;
        splay_node(t, cur); /* MUTA! */
        return fid;
    }
    cur = pageno < cur->pageno
        ? cur->left : cur->right;
    depth++;
}
```

A profundidade antes do splay é a métrica de localidade do relatório. E como o lookup rotaciona a árvore, todo leitor é escritor — o gancho da concorrência.

02 Despejo: a folha mais profunda

```
splay_evict_victim — src/splay.c

static void find_deepest(snode *n, int d,
                        int *best, snode **out) {
    if (!n) return;
    if (d > *best) { *best = d; *out = n; }
    find_deepest(n->left, d+1, best, out);
    find_deepest(n->right, d+1, best, out);
}

/* vitima e folha: sai sem rebalancear */
if (p->left == victim) p->left = NULL;
else                  p->right = NULL;
free(victim);
```

Quente sobe, frio afunda: a página menos acessada vira a folha mais profunda e sai com um free() — sem rebalanceamento.

O baseline: LRU

Hash map + lista duplamente encadeada.

- $O(1)$ lookup — o nó é encontrado direto pelo mapa
- $O(1)$ promoção — o acesso move o nó para a frente
- $O(1)$ despejo — a vítima sai da cauda da lista

Mesma interface da splay: o comparativo é justo por construção.

cache_policy.h — interface-alvo

```
typedef struct cache_policy {
    void *self;
    int      (*lookup)(void*,
                      uint64_t, int*);
    void      (*insert)(void*,
                      uint64_t, int);
    uint64_t (*evict_victim)
                (void*, int*);
    void      (*destroy)(void*);
} cache_policy_t;
```

Interface-alvo de política. Hoje o despacho é por enum: PC_POLICY_SPLAY · PC_POLICY_LRU.

Paginação, na prática

Os conceitos vistos no período, mapeados um a um no código.

Página e moldura	blocos de 4 KB do arquivo carregados em frames de RAM
Page fault $\approx$ miss	a página não está no cache; a camada busca no disco
Bit de modificação	fr->dirty marca a página escrita e ainda não persistida
Substituição	a política escolhe a vítima: splay (folha) ou LRU (cauda)
Swap-out $\approx$ write-back	a página suja é gravada no despejo, no flush e no close

```
page_cache.c — moldura + swap-out

typedef struct {
    uint64_t pageno;
    int    valid; /* moldura ocupada? */
    int    dirty; /* bit de modificacao*/
    char *bytes; /* a pagina em RAM */
} frame_t;

/* despejo: o "swap-out" da vitima */
uint64_t vpage = pol_evict(s, &vfid);
if (vf->dirty)
    disk_write_block(pc->disk,
                     vpage, vf->bytes);
vf->valid = 0;
```


Threads e exclusão mútua

Threads (pthreads)

test_concurrency sobe 8 threads com working set maior que a capacidade — despejo e releitura concorrentes. O bench mede a vazão de 1 a 16 threads.

Um mutex por shard

A região crítica é o access_page inteiro: lookup, despejo, write-back e inserção acontecem sob o mesmo pthread_mutex_t.

Por que shard

O lookup da splay rotaciona a árvore: todo leitor é escritor. A saída é particionar — shard = pageno % N — em vez de um lock global.

page_cache.c — regiao critica

```
typedef struct {
    pthread_mutex_t mu; /* 1 por shard */
    splay_t *splay;    lru_t *lru;
    frame_t *frames;   size_t nframes;
} shard_t;

static int access_page(...) {
    shard_t *s =
        &pc->shards[pageno % pc->nshards];
    pthread_mutex_lock(&s->mu);
    /* hit/miss, despejo, write-back */
    pthread_mutex_unlock(&s->mu);
    return rc;
}
```

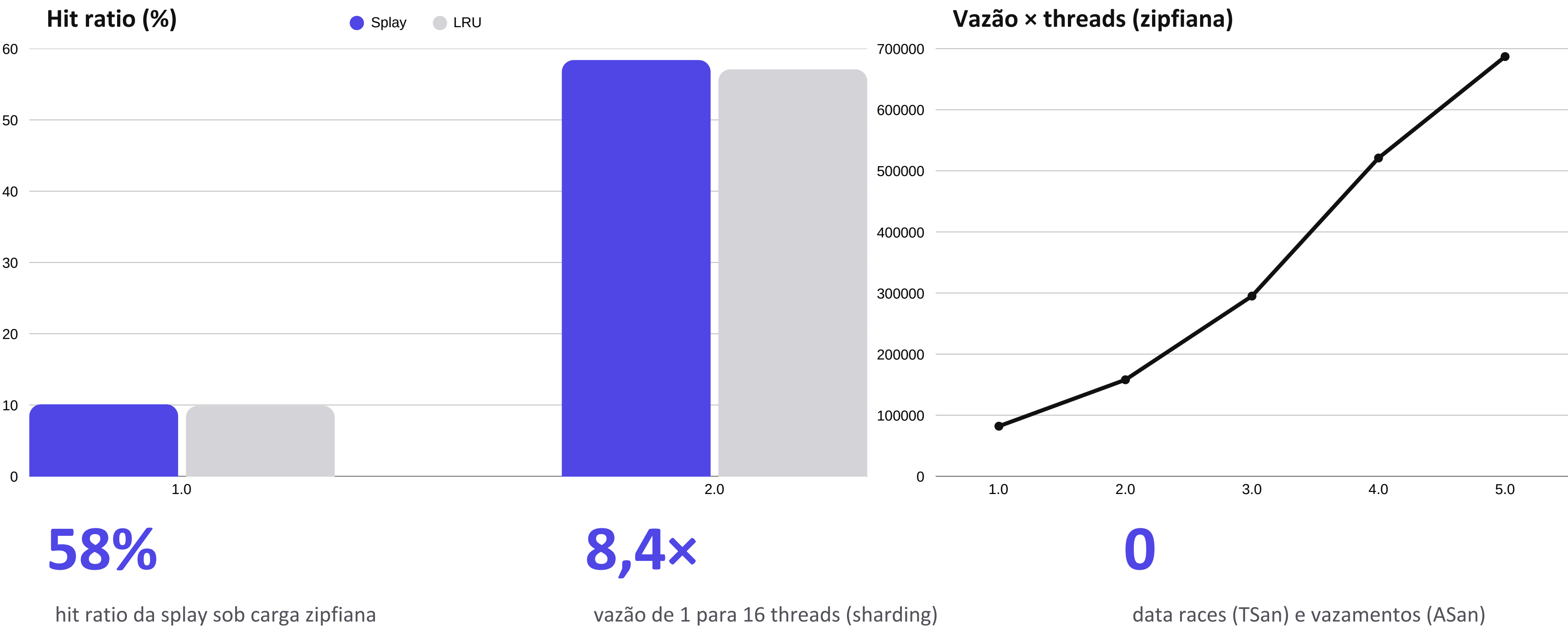
make stress roda tudo sob ThreadSanitizer — zero data races.

Cinco módulos, um contrato

page_cache	fachada estilo syscall	<code>pc_open · pc_read · pc_write · pc_flush · pc_close</code>
splay	índice autoajustável (ED)	<code>splay_lookup · splay_insert · splay_evict_victim</code>
lru	baseline do comparativo	<code>lru_lookup · lru_insert · lru_evict_victim</code>
disk	blocos com I/O posicionado	<code>pread · pwrite · contadores atômicos</code>
stats	métricas por execução	<code>hits · misses · profundidade média</code>

make test · make asan · make stress → zero warnings · zero leaks · zero data races

Splay × LRU, medido



minicache, em cinco pontos

- | | | |
|----|-------------------------------|---|
| 01 | Splay como índice vivo | o auto-ajuste leva páginas quentes à raiz — $O(\log n)$ amortizado. |
| 02 | Paginação de verdade | moldura, page fault, dirty bit e write-back — o curso, em código. |
| 03 | Threads e mutex | concorrência fina sem RW-lock: um <code>pthread_mutex</code> por shard. |
| 04 | Engenharia limpa | TDD, ASan e TSan — zero warnings, leaks e data races. |
| 05 | Tese comprovada | sob localidade zipfiana, a splay supera o LRU. Medido, não suposto. |